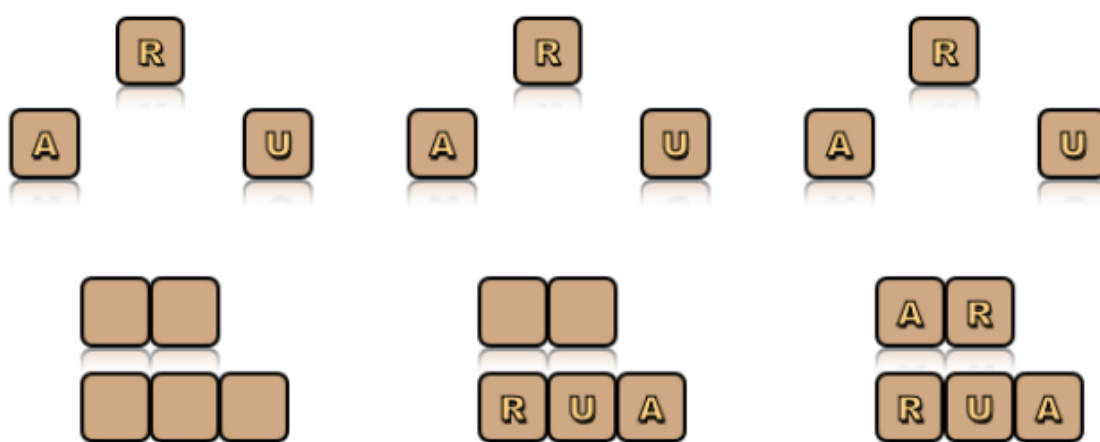


Relatório sobre o trabalho do Jogo de Palavras GURU de Programação 2



Universidade: Colégio Luís António Verney (Universidade de Évora)

Curso: Engenharia Informática

Identificações: Rafael Teixeira N: 59100, David Ramalho N:57823,
Francisco Rodrigues N:59119

Ano: 2024/25

ÍNDICE

1-Introdução	3
2-Layout	4
Primeira parte (inicial):	4
Segunda parte:	5
Funcionalidades dos Botões:	5
Informações adicionais:	5
Terceira parte (final):	6
3-Desenvolvimento do projeto (java)	7
4-Conclusão	16

1-Introdução

O jogo "Guro2P2" é um projeto desenvolvido por Rafael, Francisco e David como parte de um trabalho académico para Programação 2, implementado em Java, focado em adivinhar palavras com base nas letras mostradas.

Os jogadores recebem um conjunto de letras e devem formar palavras a partir delas.

2-Layout

Primeira parte (inicial):



Na fase inicial do jogo, o jogador é recebido por uma tela de boas-vindas que exibe o título "Guro2P2", apresentando de forma clara o objetivo do jogo. Além disso, há dois botões principais: um para **iniciar o jogo**, que leva o jogador para a fase de adivinhação de palavras, e outro para **sair**, permitindo ao usuário encerrar o jogo a qualquer momento

Segunda parte:

Moedas: 0 **Salvar** **Carregar** **Dica (100 Moedas)** **Sair**

Letras

A **R** **U**

Palavras

— —

— — —

Digite uma palavra:

Ao clicar em "Jogar", somos apresentados a esta tela onde o objetivo é formar palavras neste caso utilizando as letras disponíveis: **A**, **R** e **U**.

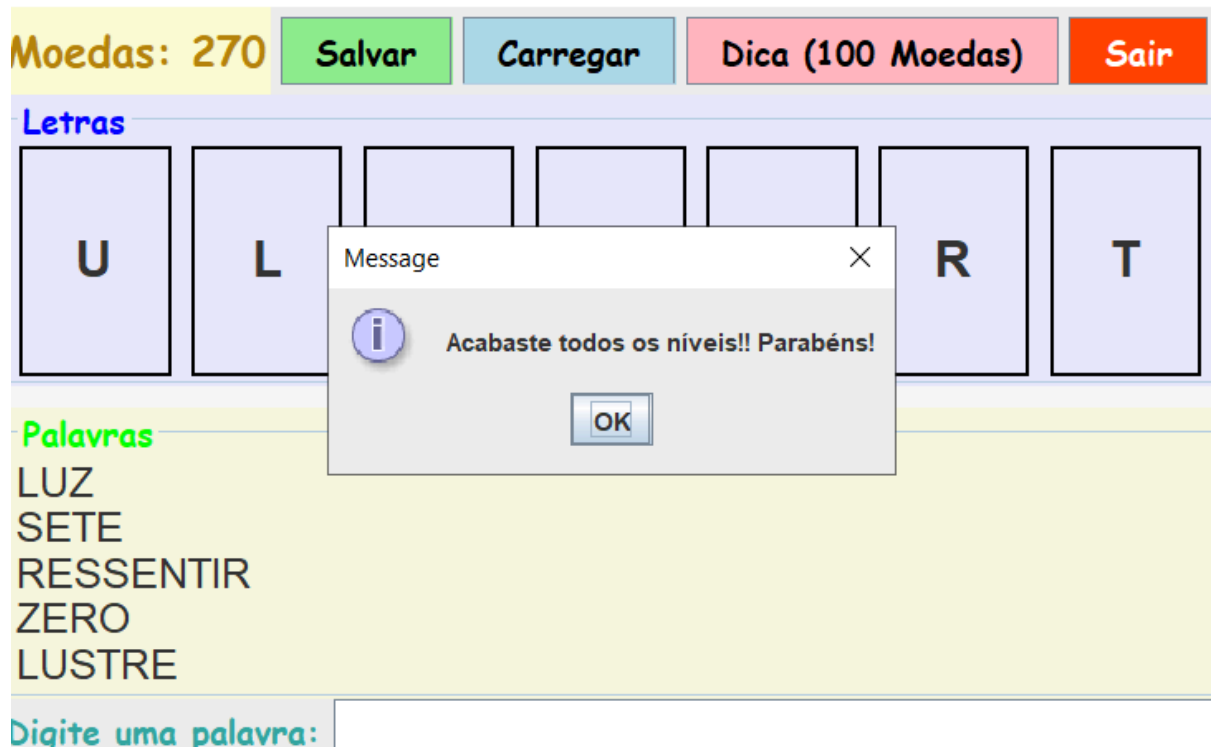
Funcionalidades dos Botões:

- **Salvar:** Permite gravar o progresso atual do jogo.
- **Carregar:** Carregar o progresso salvo anteriormente.
- **Dica (100 Moedas):** Se tivermos ao menos 100 moedas, este botão fornece a primeira letra de uma palavra como ajuda.
- **Sair:** Fecha o jogo.

Informações adicionais:

- O contador no topo esquerdo exibe o número de moedas acumuladas.
- No campo "Digite uma palavra", o jogador insere as palavras formadas pelas letras disponíveis para formar palavras e assim passar de nível

Terceira parte (final):



Por último, ao completar o último nível será mostrado uma mensagem a parabenizar e a informar que acabou o jogo.

3-Desenvolvimento do projeto (java)

Construtor GURU2P2(String levelsFile, String dictionaryFile)

Inicia o jogo carregando os níveis e o dicionário de palavras válidas a partir dos arquivos dados. Também inicializa as listas e conjuntos necessários e carrega os dados do primeiro nível.

```
1 public GURU2P2(String levelsFile, String dictionaryFile) throws IOException {
2     this.allLevels = loadLevels(levelsFile);
3     this.dictionary = loadDictionary(dictionaryFile);
4     this.guessedWords = new HashSet<>();
5     this.validExtraWords = new HashSet<>();
6     this.currentLevelIndex = 0;
7     this.coins = 0;
8     loadLevelData(currentLevelIndex);
9 }
```



Método loadLevels(String filename)

Carrega os níveis a partir de um arquivo em que cada nível é separado por uma linha em branco. Retorna uma lista de strings, cada uma representando um nível.

```
1 private List<String> loadLevels(String filename) throws IOException {
2     List<String> levels = new ArrayList<>();
3     try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
4         StringBuilder level = new StringBuilder();
5         String line;
6         while ((line = br.readLine()) != null) {
7             if (line.trim().isEmpty()) {
8                 if (level.length() > 0) {
9                     levels.add(level.toString().trim());
10                    level.setLength(0);
11                }
12            } else {
13                level.append(line).append("\n");
14            }
15        }
16        if (level.length() > 0) {
17            levels.add(level.toString().trim());
18        }
19    }
20    return levels;
21 }
```


Método loadDictionary(String filename)

Carrega o dicionário, retornando um conjunto de strings, cada uma representando uma palavra válida.

```
1 private Set<String> loadDictionary(String filename) throws IOException {
2     Set<String> words = new HashSet<>();
3     try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
4         String line;
5         while ((line = br.readLine()) != null) {
6             words.add(line.trim().toUpperCase());
7         }
8     }
9     return words;
10 }
```

Método loadLevelData(int levelIndex)

Carrega os dados do nível especificado pelo índice. Atualiza as letras atuais, as palavras do nível, e inicializa o mapa de dicas para cada palavra.

```
1 private void loadLevelData(int levelIndex) {
2     if (levelIndex >= allLevels.size()) {
3         JOptionPane.showMessageDialog(frame, "Acabaste todos os níveis!! Parabéns!");
4         return;
5     }
6
7     String[] levelData = allLevels.get(levelIndex).split("\n");
8     currentLetters = levelData[0].replaceAll("\\s+", "");
9     currentLevelWords = Arrays.asList(Arrays.copyOfRange(levelData, 1, levelData.length));
10    guessedWords.clear();
11    validExtraWords.clear();
12
13    // Inicializa o mapa de dicas para cada palavra
14    hintsIndexMap = new HashMap<>();
15    for (String word : currentLevelWords) {
16        hintsIndexMap.put(word, 0); // Índice inicial de cada palavra é 0
17    }
18 }
```



Método createAndShowGUI()

Cria e exhibe a interface gráfica inicial do jogo, incluindo o painel de início com o nome do jogo e botões para iniciar ou sair do jogo.

```
1 private void createAndShowGUI() {
2     frame = new JFrame("GURU2P2");
3     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4     frame.setLayout(new BorderLayout());
5
6     // Painel inicial com o nome do jogo e botões
7     JPanel startPanel = new JPanel();
8     startPanel.setLayout(new BorderLayout());
9     startPanel.setBackground(new Color(135, 206, 235)); // Azul claro
10
11    // Configurando o título com uma fonte personalizada
12    JLabel titleLabel = new JLabel("GURU2P2", SwingConstants.CENTER);
13    titleLabel.setFont(new Font("Comic Sans MS", Font.BOLD, 60)); // Fonte mais infantil
14    titleLabel.setForeground(new Color(255, 69, 0)); // Vermelho alaranjado
15
16    // Painel de botões
17    JPanel buttonPanel = new JPanel();
18    buttonPanel.setBackground(new Color(255, 223, 186)); // Fundo pastel
19
20    JButton startButton = new JButton("Jogar Jogo");
21    startButton.setFont(new Font("Comic Sans MS", Font.BOLD, 24));
22    startButton.setForeground(Color.WHITE);
23    startButton.setBackground(new Color(50, 205, 50)); // Verde claro
24    startButton.setFocusPainted(false);
25    startButton.addActionListener(e -> startGame());
26
27    JButton exitButton = new JButton("Sair");
28    exitButton.setFont(new Font("Comic Sans MS", Font.BOLD, 24));
29    exitButton.setForeground(Color.WHITE);
30    exitButton.setBackground(new Color(220, 20, 60)); // Vermelho carmim
31    exitButton.setFocusPainted(false);
32    exitButton.addActionListener(e -> System.exit(0));
33
34    buttonPanel.add(startButton);
35    buttonPanel.add(exitButton);
36
37    startPanel.add(titleLabel, BorderLayout.CENTER);
38    startPanel.add(buttonPanel, BorderLayout.SOUTH);
39
40    frame.add(startPanel, BorderLayout.CENTER);
41
42    frame.setSize(600, 400);
43    frame.setVisible(true);
44 }
```

Método updateLettersPanel()

Atualiza o painel de letras exibindo as letras atuais do nível.

```
1 private void updateLettersPanel() {
2     lettersPanel.removeAll();
3     for (char c : currentLetters.toCharArray()) {
4         JLabel letterLabel = new JLabel(String.valueOf(c));
5         letterLabel.setFont(new Font("Arial", Font.BOLD, 24));
6         letterLabel.setBorder(BorderFactory.createLineBorder(Color.BLACK, 2)); // Adicionando uma borda
7         lettersPanel.add(letterLabel);
8         letterLabel.setHorizontalAlignment(SwingConstants.CENTER);
9         letterLabel.setVerticalAlignment(SwingConstants.CENTER);
10    }
11    lettersPanel.revalidate();
12    lettersPanel.repaint();
13 }
```

Método updateWordsPanel()

Atualiza o painel de palavras exibindo as palavras adivinhadas ou traços(-) com o tamanho das palavras ainda não adivinhadas.

```
1 private void updateWordsPanel() {
2     wordsPanel.removeAll();
3     for (String word : currentLevelWords) {
4         JLabel wordLabel = new JLabel(guessedWords.contains(word) ? word : "_ ".repeat(word.length()));
5         wordLabel.setFont(new Font("Arial", Font.PLAIN, 20));
6         wordsPanel.add(wordLabel);
7     }
8     wordsPanel.revalidate();
9     wordsPanel.repaint();
10 }
```



Método `canFormWord(String word, String letters)`

Verifica se uma palavra pode ser formada com as letras atuais do nível.

```
1  private boolean canFormWord(String word, String letters) {  
2      Set<Character> letterSet = new HashSet<>();  
3      for (char c : letters.toCharArray()) {  
4          letterSet.add(c);  
5      }  
6  
7      for (char c : word.toCharArray()) {  
8          if (!letterSet.contains(c)) {  
9              return false;  
10         }  
11     }  
12     return true;  
13 }
```



Método `handleWordInput()`

Lida com a entrada de palavras do jogador, verificando se a palavra é válida e atualizando o estado do jogo conforme necessário.

```
1 private void handleWordInput() {
2     String input = wordInputField.getText().toUpperCase().trim();
3     wordInputField.setText(""); // Limpar o campo após cada entrada
4     if (!input.isEmpty()) {
5         if (currentLevelWords.contains(input) && !guessedWords.contains(input)) {
6             guessedWords.add(input);
7             coins += 10;
8             coinsLabel.setText("Moedas: " + coins);
9             updateWordsPanel();
10
11             // Remove a palavra do mapa de dicas
12             hintsIndexMap.remove(input);
13
14             // Verificar se todas as palavras foram adivinhadas
15             if (guessedWords.size() == currentLevelWords.size()) {
16                 JOptionPane.showMessageDialog(frame, "Nível Completo!");
17                 currentLevelIndex++;
18                 loadLevelData(currentLevelIndex); // Carregar os dados do próximo nível
19                 updateLettersPanel();
20                 updateWordsPanel();
21             }
22         } else if (dictionary.contains(input) && canFormWord(input, currentLetters)
23                 && !guessedWords.contains(input)) {
24             if (!validExtraWords.contains(input)) {
25                 validExtraWords.add(input);
26                 coins += 10;
27                 coinsLabel.setText("Moedas: " + coins);
28                 JOptionPane.showMessageDialog(frame,
29                     "Palavra válida, mas não está na lista do nível. Você ganhou 10 moedas!");
30             } else {
31                 JOptionPane.showMessageDialog(frame, "Você já usou essa palavra antes!");
32             }
33         } else if (guessedWords.contains(input)) {
34             JOptionPane.showMessageDialog(frame, "Você já usou essa palavra antes!");
35         } else {
36             JOptionPane.showMessageDialog(frame, "Palavra incorreta ou inválida.");
37         }
38     }
39 }
```

Método `saveGame(String saveFile)`

Salva o estado atual do jogo em um arquivo chamado `savegame.txt`.

```
1 private void saveGame(String saveFile) {
2     try (PrintWriter writer = new PrintWriter(new FileWriter(saveFile))) {
3         writer.println(currentLevelIndex);
4         writer.println(coins);
5         writer.println(String.join(",", guessedWords));
6         writer.println(String.join(",", validExtraWords));
7         JOptionPane.showMessageDialog(frame, "Jogo salvo com sucesso!");
8     } catch (IOException e) {
9         JOptionPane.showMessageDialog(frame, "Erro ao salvar o jogo: " + e.getMessage());
10    }
11 }
```

Método `loadGame(String saveFile)`

Carrega o estado do jogo a partir de um arquivo salvo chamado `savegame.txt`, carregando o índice do nível, as palavras acertadas, palavras validadas e quantas moedas tem.

```
1 private void loadGame(String saveFile) {
2     try (BufferedReader reader = new BufferedReader(new FileReader(saveFile))) {
3         currentLevelIndex = Integer.parseInt(reader.readLine());
4         coins = Integer.parseInt(reader.readLine());
5         String guessed = reader.readLine();
6         String extra = reader.readLine();
7         loadLevelData(currentLevelIndex);
8         if (guessed != null && !guessed.isEmpty()) {
9             guessedWords = new HashSet<>(Arrays.asList(guessed.split(",")));
10        }
11        if (extra != null && !extra.isEmpty()) {
12            validExtraWords = new HashSet<>(Arrays.asList(extra.split(",")));
13        }
14        coinsLabel.setText("Moedas: " + coins);
15        updateLettersPanel();
16        updateWordsPanel();
17    } catch (IOException e) {
18        JOptionPane.showMessageDialog(frame, "Erro ao carregar o jogo: " + e.getMessage());
19    }
20 }
```



Método giveHint()

Fornece uma dica ao jogador, revelando uma letra de uma palavra não adivinhada, em troca de moedas.

```
1 private void giveHint() {
2     if (coins >= 100) {
3         coins -= 100;
4         coinsLabel.setText("Moedas: " + coins);
5
6         for (String word : currentLevelWords) {
7             if (!guessedWords.contains(word)) {
8                 // Obtém o índice de dica para a palavra
9                 int hintIndex = hintsIndexMap.get(word);
10
11                 // Mostra a dica ao jogador
12                 JOptionPane.showMessageDialog(frame,
13                     "Dica: A " + (hintIndex + 1) + "ª letra de uma palavra é '" + word.charAt(hintIndex) + "'");
14
15                 // Atualiza o índice de dica no mapa
16                 hintsIndexMap.put(word, (hintIndex + 1) % word.length());
17                 return;
18             }
19         }
20         JOptionPane.showMessageDialog(frame, "Todas as palavras já foram adivinhadas!");
21     } else {
22         JOptionPane.showMessageDialog(frame, "Moedas insuficientes para pedir uma dica.");
23     }
24 }
```

Método main(String[] args)

Método principal que inicia o jogo, carregando os níveis, o dicionário, e exibindo a interface gráfica inicial.

```
1 public static void main(String[] args) {
2     try {
3         GURU2P2 game = new GURU2P2("ficheiro_niveis.txt", "portuguese-large.txt");
4         game.createAndShowGUI();
5     } catch (IOException e) {
6         JOptionPane.showMessageDialog(null, "Erro carregando os níveis: " + e.getMessage());
7     }
8 }
```

4-Conclusão

Concluimos o projeto com sucesso, entregamos um jogo interativo e funcional para os utilizadores. Com funcionalidades intuitivas como salvar progresso, carregar dados anteriores e a opção de obter dicas ao gastar moedas, proporcionamos uma experiência agradável e envolvente.